Course: CMPSC 132 Programming Languages

Project Title: PLang v2.0

Group Members:

Justine Dwight Acoba
John Benedict Alberto
Micko Gabriel D. Permison
Bryan Leo Udani

Project Overview

PLang v2.0 is a small, purpose-built arithmetic programming language. It is a ***hybrid*** language: every arithmetic operation can be written either with a familiar mathematical symbol (+, -, *, /, //, %, ^) or with a natural-English keyword (add, sub, mul, div, mod, pow). Both forms are parsed to exactly the same Abstract Syntax Tree (AST) and produce exactly the same result.

PLang is more than a single-expression calculator. It supports multi-statement script files, named variables with reassignment and compound assignment, nested expressions, built-in mathematical functions,and a runtime symbol table.

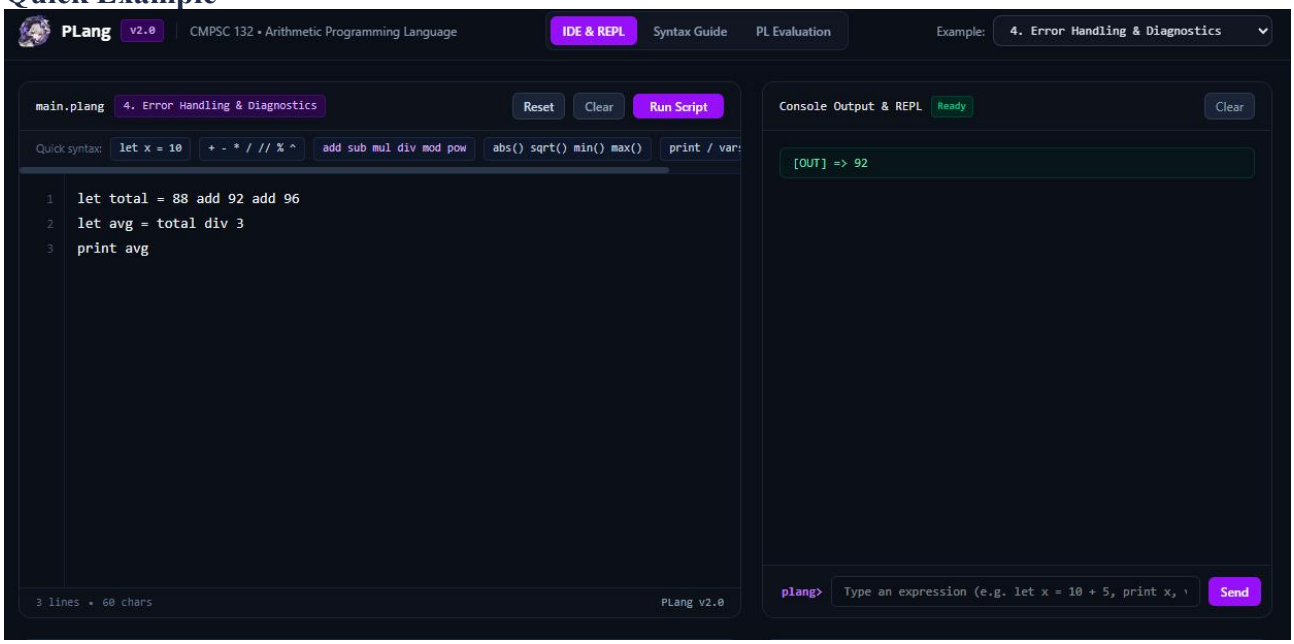
The language is available through two front ends:

1. **Python CLI and REPL** (main.py with the engine in lang.py), which runs script files or an interactive prompt.
2. **React Web Studio IDE** (web/), a browser-based implementation of the same language specification, written in JavaScript (web/src/engine/interpreter.js).

   -> https://p-lang-eight.vercel.app/

Host language versus target language. The interpreter is written in Python, but Python is only the *host implementation language*, in the same way that C is the host language for CPython. PLang is the *target language*: it has its own vocabulary, grammar, AST, semantics, and runtime symbol table, and it never calls Python's eval(). For example, 2 ^ 3 is exponentiation in PLang (result 8), whereas it is bitwise XOR in Python (result 1).

Quick Example



#	General Requirement	How PLang v2.0 Satisfies It	Where in the Code
1	Processes arithmetic operations (+, -, *, /, //, %, ^ and the keywords add, sub, mul, div, mod, pow) with exact integer preservation.	The lexer maps each keyword to the same token type as its symbol. Integer literals are parsed as Python int, and //, %, +, -, *, ^ on integers return integers. / returns a float (7 / 2 = 3.5). sqrt() returns an int for perfect squares (sqrt(25) = 5).	tokenize(), CANONICAL_TOKEN_MAP, Interpreter._binary_op_dispatch, Interpreter._eval_sqrt
2	Handles lexical, syntax (incorrect format), and runtime errors with line/column numbers, visual caret (^) pointers, and hints.	Three exception classes (LexerError, ParseError, InterpreterError) carry line, column, source line, and hint. format_error() prints a header, the offending line, a caret under the error column, and a Hint: line.	PlangError hierarchy, format_error()
3	Presents accurate outputs obeying PEMDAS/BODMAS precedence and right-associative exponentiation (2 ^ 3 ^ 2 = 512).	Precedence is encoded in the structure of the recursive-descent parser (parse_expr → parse_term → parse_exponent → parse_unary → parse_primary). parse_exponent recurses on its right operand, making ^ right-associative.	Parser.parse_expr, parse_term, parse_exponent, parse_unary, parse_primary
4	Follows Robert W. Sebesta's Programming Language Evaluation Criteria (Readability, Writability, Reliability).	See Section II for criterion-by-criterion evidence.	Entire project
5	Complete source code documentation.	Every module, class, and method has a docstring, and every class documents its class variables and instance attributes. See Section V.	lang.py, main.py

IN SHORT Sa PLang, kung ano ang sinabi, yun talaga ang ibig sabihin. Ang 2 ^ 3 ay 8, hindi 1. Hindi tulad ng "wala yun" na ang totoong meaning pala ay "meron talaga."

Concepts of Programming Languages evaluates a language mainly by its **readability**, **writability**, and **reliability**. The following subsections show how each criterion is addressed in PLang v2.0.

Readability

Readability is the ease in which programs can be read and understood.

Feature	Description	Example
Hybrid symbolic/English operators	Users may use **symbols or self-documented words.** div and mod remove the ambiguity of / versus // and the "percentage" misreading of %.	47 div 10 → 4; 47 mod 10 → 7; 20 add 5 → 25
Explicit let declarations	let makes a new variable binding obvious without type keywords or semicolons. The shorter x = 10 form is also accepted. Just like existing programming languages	let score = 100
Clear statement keywords	Statements begin with a descriptive word. Just like existing programming languages	print, vars, clear
vars symbol table inspector	One command prints every variable with its name, data type, and value in a formatted table.	vars
--tokens visualizer	Prints the lexical token stream (type, lexeme, line, column).	python main.py --tokens script.plang
--ast visualizer	Prints the AST as an ASCII tree.	python main.py --ast script.plang
Simplicity and orthogonality	One unified grammar; each operator works the same way on any numeric operand.	

Writability

Writability is the ease in which a language can be used to create programs.

Feature	Description	Example
Zero boilerplate	No main(), class wrapper, headers, imports, or semicolons. A one-line file is a complete program. Straight to the point :>	print 10 + 5
Compound assignment	+=, -=, *=, /= update an existing variable.	x += 5
Built-in math functions	abs, sqrt, min, max, round, int, float, available **without the use of imports.** min and max accept two or more arguments.	max(4, 12, 2) → 12
Automatic ans variable	After each expression, print, or assignment, ans **holds the most recent value.**	ans * 2
Built-in constants	PI and E are **pre-defined.**	PI * 2
Flexible print syntax	print x and print(x) are both valid.	

Reliability

Reliability is conformance to specification under all conditions, including the handling of errors.

Feature	Description
Exact int preservation	Integer operands stay integers through +, -, *, //, %, and ^.
Strict precedence hierarchy	Parentheses, exponentiation (right-associative), multiplicative, then additive. **See Formal Language Specification.**
Constant protection	PI and E cannot be reassigned (let PI = 3 raises a runtime error).
Exponent overflow guard	Exponents with magnitude greater than MAX_EXPONENT_MAGNITUDE (10000) on bases with magnitude greater than 1 are rejected instead of hanging the interpreter. Idk how this works honestly
Three-stage error trapping	Lexical, syntax, and runtime errors are caught separately, each reported with line, column, source line, caret, and hint.
Domain and type guards	Division/modulo by zero, zero to a negative power, sqrt of a negative number, complex results, wrong argument counts, unknown functions, undefined variables, and non-numeric operands are all trapped.
Error recovery in scripts	run_file reports an error on one line and continues so every error in a file is shown; the process then exits with status 1.

Ang 47 div 10 ay malinaw kung ano ang ibig sabihin. Ang "ok lang ako" ay kailangan pang i-parse ng tatlong araw, tapos may syntax error pa rin.

Formal Language Specification

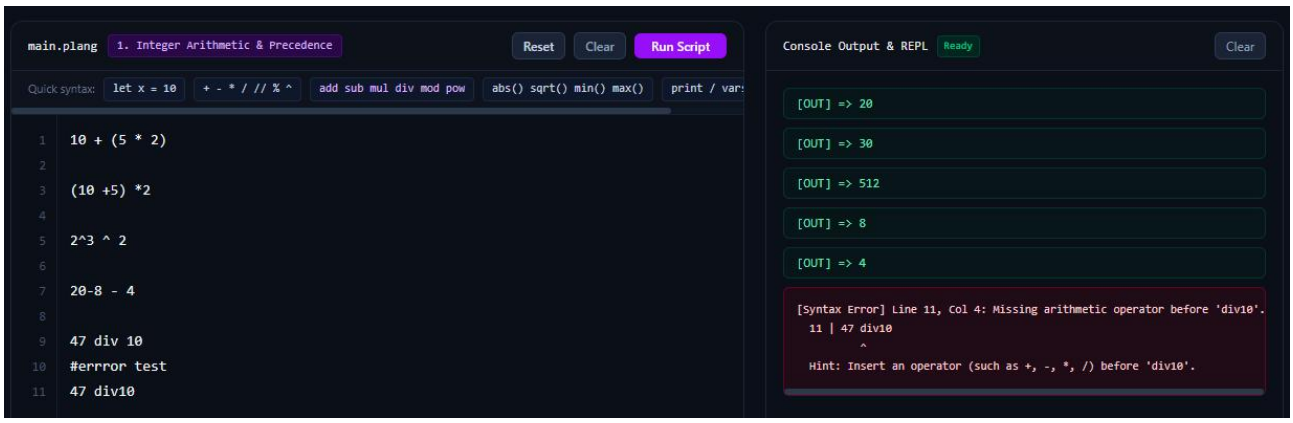
EBNF Grammar

The grammar below is taken directly from the Parser class docstring in lang.py. Each production corresponds to one parser method.

```
Program    ::= Statement* EOF
Statement  ::= LetStmt | AssignStmt | PrintStmt | ClearStmt | VarsStmt | ExprStmt
LetStmt    ::= "let" IDENT "=" Expr
AssignStmt ::= IDENT ( "=" | "+=" | "-=" | "*=" | "/=" ) Expr
PrintStmt  ::= "print" Expr
ClearStmt  ::= "clear"
VarsStmt   ::= "vars"
ExprStmt   ::= Expr
Expr       ::= Term ( ( "+" | "-" | "add" | "sub" ) Term )*
Term       ::= Exponent ( ( "*" | "/" | "//" | "%" | "mul" | "div" | "mod" ) Exponent )*
Exponent   ::= Unary ( ( "^" | "**" | "pow" ) Exponent )?
Unary      ::= ( "+" | "-" | "sub" ) Unary | Primary
Primary    ::= NUMBER | IDENT "(" ArgList? ")" | IDENT | "(" Expr ")"
```

The **reserved keywords** are let, print, clear, vars, add, sub, mul, div, mod, and pow.

Worked examples.



doesn’t matter how u arrange it :> spacing matters if hybrid/english approach

Operator Semantics

Operator(s)	Meaning	Result Type
+, add	Addition	int if both operands are int, else float
-, sub	Subtraction (binary) / negation (unary)	same rule
*, mul	Multiplication	same rule
/	True division	always float (7 / 2 = 3.5)
//, div	Floor division	int if both operands are int
%, mod	Remainder	same rule as +
^, **, pow	Exponentiation	same rule as + (an int raised to a non-negative int stays int)

Architecture and Libraries Utilized (IDK HOW IT WORKS EITHER T^T)

Three-Stage Pipeline

```
Source text ---> [Stage 1: Lexer] ---> Token stream
        tokenize()

Token stream ---> [Stage 2: Parser] ---> AST
        Parser

AST ---> [Stage 3: Interpreter] ---> Output / Symbol Table
        Interpreter + ASTNode.evaluate()/execute()
```

Stage 1: Table-Driven Regex Lexical Analyzer (tokenize)

tokenize() compiles a single master regular expression from an ordered list of named patterns. The order of the patterns matters: malformed-number and malformed-identifier patterns come before the normal ones, multi-character operators (+=, //, **) come before single-character operators, and keywords use word boundaries (\blet\b) so that an identifier such as letter is not split. Each match is routed through the TOKEN_HANDLER_DISPATCH dictionary to a small handler function: some raise a LexerError (MALFORMED_FLOAT, MALFORMED_IDENT), some skip whitespace and comments, one tracks newlines, and the default handler emits a Token. English keywords are normalized to the same token types as their symbols through CANONICAL_TOKEN_MAP (for example add becomes ADD, div becomes IDIV, pow becomes POW). Every token records its line and column. The stream ends with an EOF token.

Stage 2: Recursive-Descent Parser (Parser)

The Parser class implements one method per grammar rule and builds the AST. Statement keywords are dispatched through _statement_dispatch, and primary expressions through _primary_dispatch. Operator precedence is a direct consequence of the call chain parse_expr → parse_term → parse_exponent → parse_unary → parse_primary: operators handled deeper in the chain bind more tightly. The parser raises a ParseError with context-specific messages for problems such as missing operands, unbalanced parentheses, reserved words used as variable names, and invalid assignment targets.

Stage 3: Polymorphic AST and Table-Driven Tree-Walk Interpreter (Interpreter)

The AST is built from ten node classes. Each node knows how to evaluate itself (evaluate), run as a statement (execute), and draw itself (format_tree). The Interpreter class holds the symbol table (variables), the execution history, and dispatch tables for binary operators (_binary_op_dispatch) and built-in functions (_single_arg_funcs, _variadic_funcs). It validates operand types, applies the operator through Python's operator module, and raises InterpreterError for runtime faults.

Python Standard Libraries Utilized (Requirement III.1)

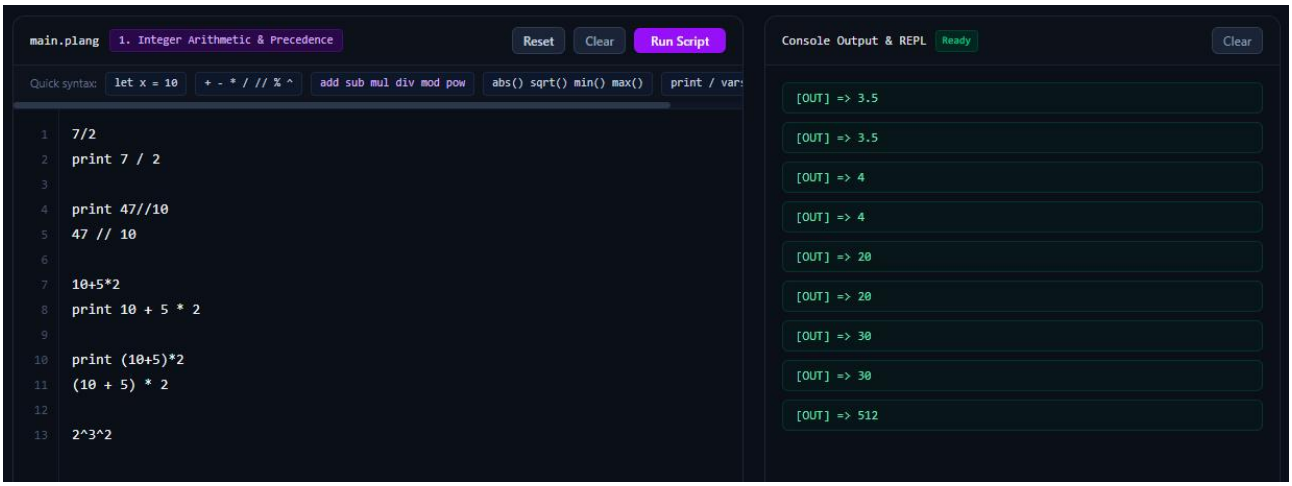
Library	Module	How It Is Used in PLang
re	lang.py	Builds the master regular expression in tokenize(), using named groups so match.lastgroup identifies the token type. re.escape() safely embeds operator characters and the configurable comment prefix.
math	lang.py	Supplies the constants PI (math.pi) and E (math.e) in Interpreter.BUILT_IN_CONSTANTS, and math.sqrt() inside _eval_sqrt.
operator	lang.py	Provides first-class arithmetic functions (operator.add, sub, mul, truediv, floordiv, mod, pow, pos, neg) stored in dispatch tables, so no if/elif chain is needed to select an operation.
json	main.py	load_config() reads config.json (banner, comment prefix, print prefix, error templates).
argparse	main.py	main() parses the script path and the --ast, --tokens, and --config flags.
os	main.py	Resolves the default configuration path (os.path.dirname, os.path.abspath, os.path.join) and checks that a script exists (os.path.exists).

Sample Execution Outputs and Error Handling Demonstration

All outputs in this section were produced by running the submitted lang.py and main.py.

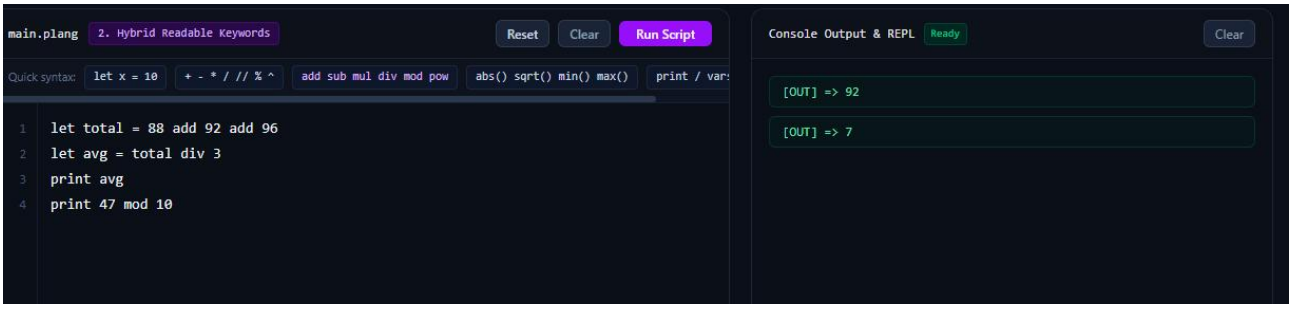
Valid Programs

Integer Arithmetic

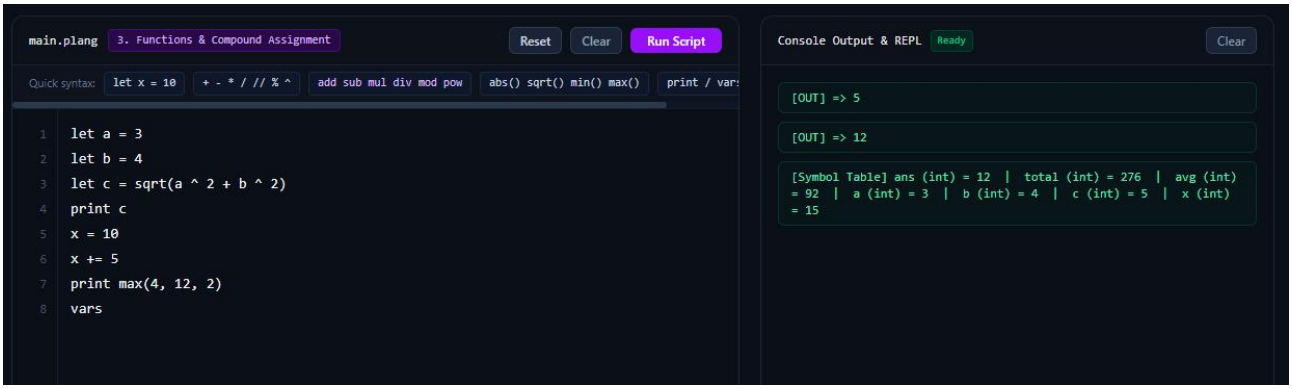


With our without using print btw

Hybrid English Keywords



Functions and the Symbol Table (vars)



sorry if that kind of output is scattered :<

Error Handling Matrix

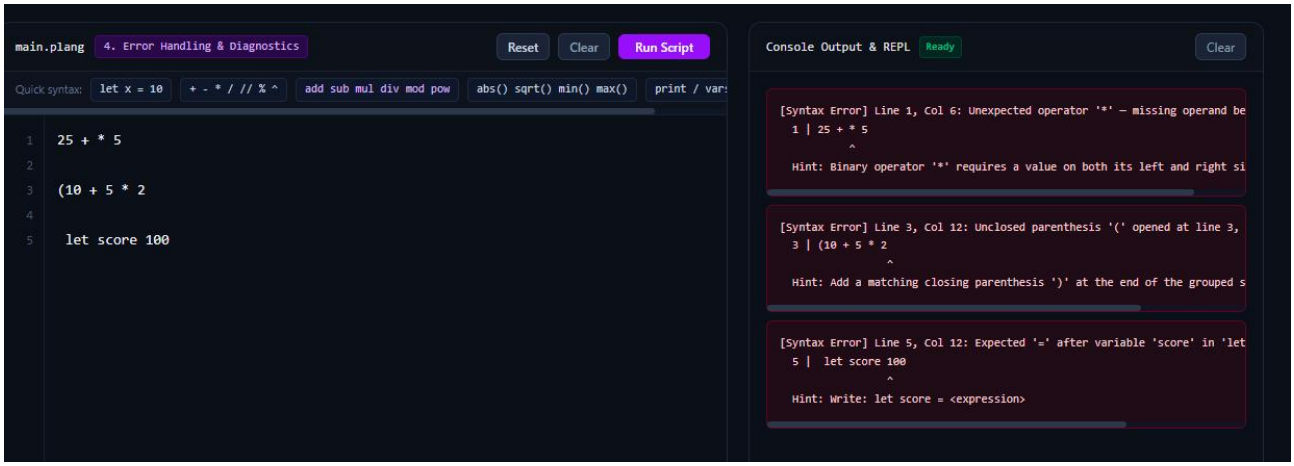
Every error report has the same four-part layout: a header with the error category, line, and column; the offending source line; a caret (^) under the error column; and a Hint: line.

Lexical Errors



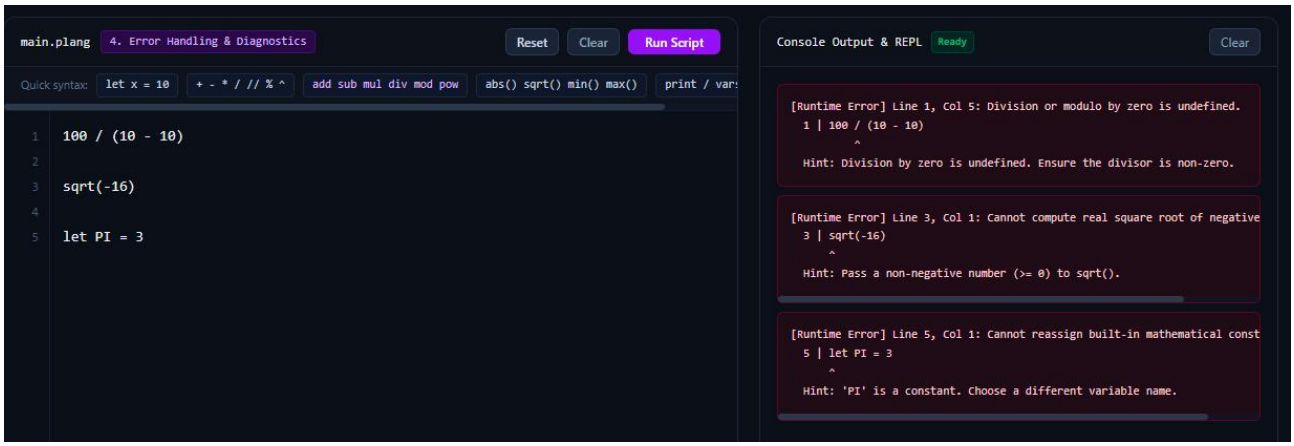
May hint nadin yan para dkana masaktan

Syntax / Incorrect-Format Errors



doesnt matter where u run it too btw some of those u see contains each of their own example pre coded inputs

Runtime Errors



Summary of Error Categories

Stage	Exception	Header Label	Typical Causes
1. Lexer	LexerError	[Lexical Error]	Unknown characters, multiple decimal points, identifiers starting with a digit
2. Parser	ParseError	[Syntax Error]	Missing operand, unbalanced parentheses, missing =, reserved word as a variable name, invalid assignment target
3. Interpreter	InterpreterError	[Runtime Error]	Division or modulo by zero, negative sqrt, undefined variable, constant reassignment, exponent overflow, wrong argument count